

# 1 Time Complexity

## 1.1 Measuring Complexity

- 定义:  $M$  是一个确定性图灵机, 确定性图灵机对所有输入都停机。确定性图灵机  $M$  的 time complexity/running time 是函数  $f: \mathcal{N} \rightarrow \mathcal{N}, f(n)$  的意思是输入图灵机的长度为  $n$ , 图灵机最多要经过多少步给出输出。如果  $f(n)$  是图灵机  $M$  的运行时间, 可以说  $M$  是一个  $f(n)$  时间的图灵机。
- Big-O and Small-o
  1. 在 asymptotic analysis(渐进分析) 中, 我们将对算法确切运行时间的复杂表达式忽略其系数以及所有低阶项, 保留时间表达式的最高阶项。
  2. 设  $\mathcal{R}^+$  是非负实数集。如果  $f$  和  $g$  是两个将  $\mathcal{N} \rightarrow \mathcal{R}^+$  的函数, 如果正整数  $c$  和  $n_0$  存在并且每一个整数  $n \geq n_0$  使得  $f(n) \leq cg(n)$ , 那么  $f(n)=O(g(n))$ 。如果  $f(n)=O(g(n))$ , 我们说  $g(n)$  是  $f(n)$  的 asymptotic upper bound。忽略常数和低阶项后,  $f(n)$  的增长速度不会超过  $g(n)$ 。
  3. Small-o notion 表示一个函数渐进意义上小于我们使用小  $o$  符号的函数。设  $\mathcal{R}^+$  是非负实数集。如果  $f$  和  $g$  是两个将  $\mathcal{N} \rightarrow \mathcal{R}^+$  的函数, 如果  $\frac{f(n)}{g(n)}$  当  $n$  趋于无穷大的时候值为 0, 那么  $f(n) = o(g(n))$ 。也就是说, 对于任意实数  $c>0$ , 存在一个数  $n_0$ , 对于所有  $n \geq n_0$  的数,  $f(n)<cg(n)$ 。
- Analyzing Algorithm
  1. 对于图灵机  $M_1$  判定语言  $A = \{0^k 1^k | k \geq 1\}$  的时间复杂度。
    - 首先扫描整个纸带, 判断是否 1 在 0 的右侧, 如果 1 在 0 的左侧直接 reject, 扫描完后指针返回,  $n+n$  次操作,  $O(n)$  时间复杂度。
    - 对于图灵机每次扫描, 如果指针扫描第一个 0 时, 需要寻找第一个 1, 每次扫描整个纸带只能删除 1 对 01, 因此扫描需要  $n/2$  次, 如果每次扫描都要扫描整个纸带的元素的话,  $(n/2)*O(n)$ , 时间复杂度  $O(n^2)$ 。

- 对于删除所有 01 对后，扫描整个纸带的元素进行判断，如果仍然存在 0 或者 1，reject. $O(n)$
- 因此该图灵机是  $O(n^2)$  的时间复杂度。
- 2. 定义：t 函数是  $\mathcal{N} \rightarrow \mathcal{R}^+$  的映射。定义 time complexity class 时间复杂度类， $\text{TIME}(t(n))$  定义为所有在  $O(t(n))$  时间图灵机可判定的所有语言集合。
- 3. 发现：对于识别语言 A 的时间复杂度，依赖于计算模型的选择。
- Complexity relationships among models
  1.  $t(n)$  是一个函数，并且  $t(n) \geq n$ 。对于多带图灵机的每一步操作，都需要单带图灵机  $O(t(n))$  步进行模拟，多带图灵机每一步需要  $t(n)$  时间，那么多带到单带  $O(t^2(n))$
  2. 定义：N 是一个 nondeterministic Turing machine 并且是一个判定器 (是判定器的非确定性图灵机中，对于所有输入，它的每一条计算分支都会停机，没有无线循环分支)。N 的运行时间是函数  $f: \mathcal{N} \rightarrow \mathcal{N}$ ，并且  $f(n)$  是 N 在输入长度为  $n$  的输入计算的任何分支上使用的最大步数。
  3. 定理：t(n) 是一个函数，并且  $t(n) \geq n$ 。每一个  $t(n)$  时间的非确定性单带图灵机有一个等价的  $2^{O(t(n))}$  time deterministic single tape Turing machine.

## 1.2 The class P P 类

- P 是在在确定性单带图灵机上能在多项式时间可判定的语言类。
- $P = \bigcup_k \text{TIME}(n^k)$
- 对于多项式等价于确定性单带图灵机的计算模型，P 类是 invariant 不变的。
- P 大概对应了在计算机上真实能够解决的问题类
- Examples of Problems in P

1. PATH 问题, 给定一个有向图, 起点  $s$ , 终点  $t$ , 判断是否存在从  $s$  到  $t$  的路径? 定理:  $PATH \in P$ , 证明 PATH 问题可以被确定性图灵机在多项式时间内判定

定义一个图灵机  $M = \langle \rangle$  在输入  $\langle G, s, t \rangle$  中,  $G$  是有向图,  $s$  是起始节点,  $t$  是终点:

- (1) 首先在起始节点  $s$  做标记
- (2) 接着反复执行以下操作直到左右节点都被标记: 每一次扫描有向图  $G$  的每一条边, 如果一条边  $\langle a, b \rangle$  发现从一个标记的节点  $a$  到未标记的节点  $b$ , 那么把  $b$  标记
- (3) 如果  $t$  被标记, 则 accept, 否则 reject。

"

2. RELPRIME 问题, 两个数是互质如果 1 是能同时整除两个数的最大整数。定理:  $RELPRIME \in P$

使用 Euclidean 算法,

$E = \langle \rangle$  On input  $\langle x, y \rangle$ ,  $x$  和  $y$  是自然数:

- (1) 首先, 执行以下算法重复  $y$  直到  $y=0$ :  $x = x \bmod y$ , exchange  $x$  and  $y$
- (2) 接着, 输出  $x$

"

- (3) 算法  $R$  解决 RELPRIME, 使用  $E$  作为子程序

$R = \langle \rangle$  On input  $\langle x, y \rangle$ ,  $x$  和  $y$  是自然数:

- (4) 运行算法  $E$ ,  $\langle x, y \rangle$  作为输入
- (5) 如果输出的值为 1, accept。否则, reject。

"

### 1.3 The Class NP NP 类

- eg1 Hamiltonian path | 哈密顿路径 | HAMPATH Problem

在一个有像图  $G$  中, 哈密顿路径是一条经过恰好一次经过每个节点的有向路径。

$HAMPATH = \langle G, s, t \rangle \mid G \text{ is a directed graph with a Hamiltonian path from } s \text{ to } t$

哈密顿路径问题具有多项式可验证性 (polynomial verifiability)。多项式可验证性意味着验证某个问题的存在性，远比判定其存在性更加容易。验证是核定给定解的有限性，而判定是通过定理推导出解的 pattern。

- eg2 合数的判定问题 | compositeness problem

若一个自然数是两个大于 1 的整数的乘积，那么这个数是合数。

$\text{COMPOSITES} = \{x \mid x = pq, \text{ for integers } p, q > 1\}$

- 定义：语言 A 的验证器是一个算法 V，

$A = \{\omega \mid V \text{ accepts } \langle \omega, c \rangle \text{ for some string } c\}$   $\omega$  是属于 A 集合的语言，c 是用于验证  $\omega$  是属于 A 集合语言的辅助额外字符串。多项式时间验证器的运行时间是关于  $\omega$  输入长度的多项式运行时间。若语言 A 存在多项式时间验证器，那么称 A 是多项式可验证的。c 是 certificate 证书。

- 定义:NP(Nondeterministic polynomial time) 是具有多项式时间验证器的语言类。P 类是 NP 类的子集。

- NTM 求解 HAMPATH

$N_1 = \langle G, s, t \rangle$  作为输入，G 是有 s 节点和 t 节点的有向图，

1. 生成一个 m 个数字的列表，m 是有向图的节点个数。每一个在列表中的数都是不确定性被选择 (为了同时覆盖所有可能) 在 1 到 m 之间。
2. 检查列表是否含有重复项，如果发现重复项，则拒绝。(淘汰重复的无效分支)
3. 检查是否  $s = p_1$  且  $t = p_m$ ，如果失败，则拒绝。(保证起点和终点符合要求)
4. 对于每一个 i 在 1 到 m-1 之间，检查是否  $(p_i, p_{i+1})$  是有向图 G 的边，如果不是则拒绝，如果全部通过则 accept。

"

- 定理: 一个语言属于 NP 类当且仅当这个语言能够被某个非确定性多项式时间图灵机所确定。

推导。我们设：A 可由多项式时间非确定性图灵机 N 判定，V 是根据 NP 类定义所存在的用于语言 A 多项式时间验证器，V 是运行时间为  $n^k$  的图灵机。

N=" 在输入长度为 n 的 w 上

1. 非确定选择字符串 c，并且 c 的长度不超过  $n^k$ 。批量生产所有可能的 certificate。 $n^k$  确保验证过程能在多项式时间内完成。
2. 运行 V 在输入  $\langle w, c \rangle$
3. 如果一个 V 分支 accept，accept；否则如果所有 V 的分支都拒绝，则拒绝。

"

V=" 在输入  $\langle w, c \rangle$ , w 和 c 均为字符串（有点像 For+if+break）

1. 在输入串 w 上模拟非确定性图灵机 N 的运行（验证器 V 是确定性图灵机，要还原 N 度非确定性计算过程）。证书 c 的每一个字符都对应着 N 在计算遇到的每一个非确定性选择点。
2. 如果 N 的计算过程分支接受，则 V 接受，否则拒绝。

"

- 定义：NTIME( $t(n)$ )=L|L is a language decided by a  $O(t(n))$  time non-deterministic Turing machine。NP= $\cup_k$  NTIME( $n^k$ )
- clique 问题。在无向图中一个 clique 是一个子图，其中在任意两个节点都有一条边相连。一个 k-clique 是一个包含 k 个节点的 clique。Clique 问题是去确定是否一个图包含指定大小的团。

CLIQUE= $\{ \langle G, k \rangle | G \text{ is an undirected graph with a } k\text{-clique} \}$

CLIQUE 属于 NP 问题。clique 是证书。

证明：

V=" 在输入为  $\langle G, k \rangle, c \rangle$ ：

1. 检验 c 是否是 G 中 k 个节点组成的集合
2. 验证是否 G 包含所有连接 c 中节点的所有边
3. 如果全部通过，则 accept；否则拒绝

" N=" 在输入为  $\langle G, k \rangle$ :

1. 非确定性地选择  $G$  中的  $k$  个节点的子集 clique
2. 验证是否  $G$  包含连接这个 clique 的所有边
3. 如果有一个通过, 则接受; 如果全部不通过, reject。

"

- P vs NP Problem

1. P 类问题是可以在多项式时间内判定某字符串是否属于该语言的语言构成的集合
2. NP 类问题是可以在多项式时间内验证某字符串是否属于该语言的语言构成的集合

#### 1.4 NP-Completeness NP 完全性

- Cook-Levin 理论: 如果某一类特定的 NP 问题存在多项式时间算法 ( $P=NP$ ), 那么所有 NP 问题都能找到多项式时间算法。SAT 是 NP-complete, 3SAT 是 NP-complete
- SAT 问题, 可满足问题, satisfiability problem。一个可满足问题是去检验是否一个布尔公式是可满足的。如果对布尔公式的变量进行 0 和 1 的赋值使这个公式值为 1 我们称这个布尔公式可满足。

$SAT = \{ \langle \phi \rangle \mid \phi \text{ is a satisfiable Boolean formula} \}$

1. 定理:  $SAT \in P$  iff  $P=NP$

- Polynomial Time Reducibility 多项式时间可归约性

1. 定义: 一个方程  $f: \Sigma^* \rightarrow \Sigma^*$  (从由字符集构成的字符串集合映射到字符集构成的字符串集合) 是一个多项式时间可计算函数, 如果某个多项式时间图灵机  $M$  存在并且对于任何输入  $w$ , 图灵机  $M$  都会在停机时在读写纸带上写下  $f(w)$ 。简单来说: 函数  $f$  是多项式时间可计算函数, 如果存在一台多项式时间图灵机  $M$  满足, 给  $M$  输入  $f$  的任意一个输入字符串,  $M$  会在多项式时间内停机, 并且停机时, 它的读写带上只有  $f(w)$  这一个字符串输出。

2. 定义: 若存在一个多项式时间可计算函数  $f: \Sigma^* \rightarrow \Sigma^*$ , 使得对每一个  $w$  存在  $w \in A \iff f(w) \in B$ , 则称语言  $A$  多项式时间映射可归约到语言  $B$ , 记作  $A \leq_P B$ .

3. 定理: 如果语言  $A$  多项式时间可归约到  $B$ , 并且  $B$  属于  $P$  类, 那么  $A$  属于  $P$  类。if  $A \leq_P B$  and  $B \in P$ , then  $A \in P$ .

假设  $M$  是判定  $B$  的多项式时间算法,  $f$  是从  $A$  到  $B$  的多项式时间归约, 我们描述一个多项式时间算法  $N$  判定  $A$ :

$N$ : “对于输入  $w$

(a) 计算  $f(w)$

(b) 使用  $f(w)$  作为输入, 运行  $M$ , 输出  $M$  输出的内容。

”

4. A literal 字面量

(a) 字面量是布尔表达式的变量及其取反的布尔变量。

(b) clause (子句) 是布尔变量使用析取符号 (并集) 进行连接的表达式子。如果若干子句通过合取符号 (交集) 进行连接的表达式子, 我们称之为合取范式 (conjunctive normal form/cnf).

(c) 如果有 3 个子句构成的合取范式, 则称为 3 cnf-formula。

(d) 定理: 首先我们定义一个  $3SAT = \{ \langle \phi \rangle \mid \phi \text{ is a satisfiable 3 cnf-formula} \}$  在可满足的 3 合取范式中, 每一个 clause 都包含至少一个被赋值为 1 的字面量。 **3SAT is polynomial time reducible to CLIQUE.**

证明的关键是构造一个多项式时间转换函数  $f$ , 把  $3SAT$  的输入 (公式  $\phi$ ) 转换成  $CLIQUE$  的输入且满足:  $\phi$  是 satisfiable 能够推出图  $G$  中存在大小为  $k$  的团,  $\phi$  是 unsatisfiable 能够推出图  $G$  中不存在大小为  $k$  的团

i. 假设  $\phi$  是一个带有  $k$  个子句的函数:  $\phi = (a_1 \vee b_1 \vee c_1) \wedge (a_2 \vee b_2 \vee c_2) \wedge \dots \wedge (a_k \vee b_k \vee c_k)$ 。把  $k$  个子句拆成 3 个节点的  $k$  组三元组, 并从  $k$  组节点中挑选  $k$  个节点, 要满足: 同一个三元组内节点不相连 (SAT 要求每个子句满足赋值后结果为 1, 因此在转换时我们要从  $k$  个子句中每一个子句仅挑选一个节点), 矛盾字面量不相连 (如果一

个组中  $x_k=1$ , 那么  $x_k$  取反在另一个子句中出现要是 0, 这样  $x_k$  就矛盾了。)

- NP 完全性的定义

1. 定义: 一个语言是 NP-complete 如果满足两个条件

- (a) B 是在 NP 类中

- (b) 在 NP 类中的 A 是多项式时间归约到 B

2. 定理: 如果 B 是 NP-complete 并且 B 属于 P, 那么  $P=NP$ 。

证明: 根据 B 的定义, 每一个在 NP 类中的 A 是多项式时间归约到 B, B 又属于 P 类, 那么 A 同样属于 P 类, 根据 Cook-Levin 定理, 推出  $P=NP$ 。

3. 定理: 如果 B 是 NP-complete 并且  $B \leq_P C$  对于 NP 类中的 C, 那么 C 是 NP-complete。

证明: 我们已知 B 是 NP 完全, 那么 NP 类中的 A 都可以在多项式时间归约到 B, 由于语言 B 又可以在多项式时间内归约到 C, 因此 A 可以在多项式时间内归约到 C, C 是 NP-complete。

## 1.5 Additional NP-Complete Problem

- 在构造从 3SAT 问题到某一语言的多项式时间归约时, 我们需要寻找该语言中能够模拟布尔公式里变量和子句的结构, 这种结构叫做 gadget(构件)。

- Vertex Cover Problem 顶点覆盖问题

若 G 为无向图, G 的顶点覆盖是满足如下条件的顶点子集: 图 G 的每条边都至少与该子集中的一个顶点相关联。简单来说, 就是图 G 的所有边占有的不同的图 G 中的节点。  $VERTEX-COVER = \{ \langle G, k \rangle \mid \exists \text{ a set of } k \text{ nodes in } G \text{ such that every edge in } G \text{ has at least one endpoint in the set} \}$ 。



一、先把问题简化

想象一个更简单的场景：

**3SAT公式：**

text 复制 下载

$\phi = (x \vee y)$  - 只有一个子句、两个变量

**构造的图：**

1. 变量组件：

text 复制 下载

$x \rightarrow \neg x$   
 $y \rightarrow \neg y$

2. 子句组件（因为只有一个子句  $(x \vee y)$ ，我们需要三个文字，假设是  $(x \vee y \vee z)$ ，但为了简单，我们先理解原理）

图 1: Vertex-Cover1-1

二、证明的第一部分：如果 $\phi$ 可满足  $\Rightarrow$  图有顶点覆盖

**假设赋值：x=TRUE, y=FALSE**

这使  $\phi = (x \vee y)$  为真（因为x为真）。

**构造顶点覆盖：**

1. 选变量节点：根据赋值选对应的节点

- $x = \text{TRUE} \rightarrow$  选节点  $x$ （不选  $\neg x$ ）
- $y = \text{FALSE} \rightarrow$  选节点  $\neg y$ （不选  $y$ ）
- $\rightarrow$  用了 2 个顶点

2. 选子句节点：对每个子句，选另外两个节点

- 子句  $(x \vee y)$ ：x 为真  $\rightarrow$  不选子句中的  $x$  节点，选  $y$  节点和（如果有第三个文字）第三个节点
- $\rightarrow$  用了 2 个顶点

图 2: Vertex-Cover1-2

**为什么这能覆盖所有边？**

1. 变量边：  $x \rightarrow \neg x$  被  $x$  覆盖，  $y \rightarrow \neg y$  被  $\neg y$  覆盖  $\checkmark$

2. 三角形内部边：被选的2个子句节点覆盖  $\checkmark$

3. 连接边：

- 子句  $x \rightarrow$  变量  $x$ ：被变量  $x$  覆盖（因为  $x$  为真）
- 子句  $y \rightarrow$  变量  $y$ ：被子句  $y$  覆盖（因为  $y$  为假，我们选了子句  $y$  节点）

**三、证明的第二部分：如果图有顶点覆盖  $\Rightarrow \phi$  可满足**

**假设图有一个大小为  $k$  的顶点覆盖**

其中  $k = m + 2l$  ( $m$  个变量， $l$  个子句)

**顶点覆盖必须包含：**

1. 每个变量边至少1个节点（否则那条边没被覆盖）

2. 每个三角形至少2个节点（否则三角形内部有边没被覆盖）

图 3: Vertex-Cover1-3

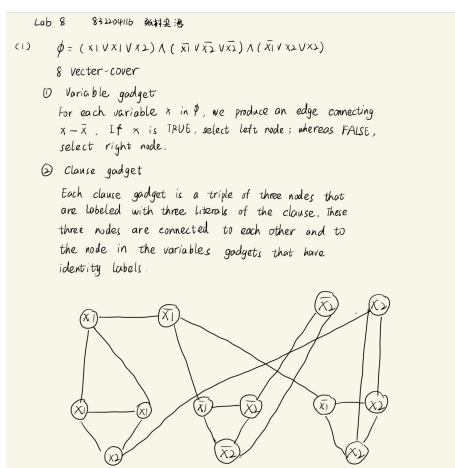


图 4: Vertex-Cover2-1

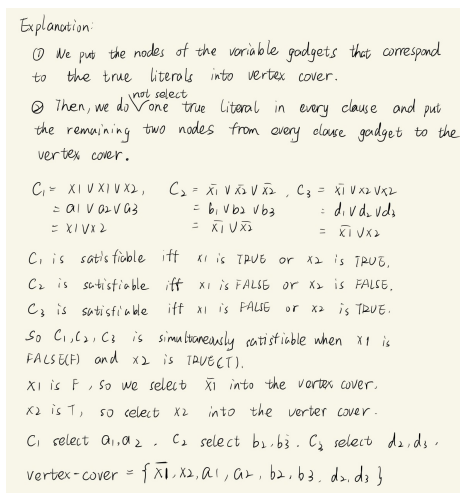
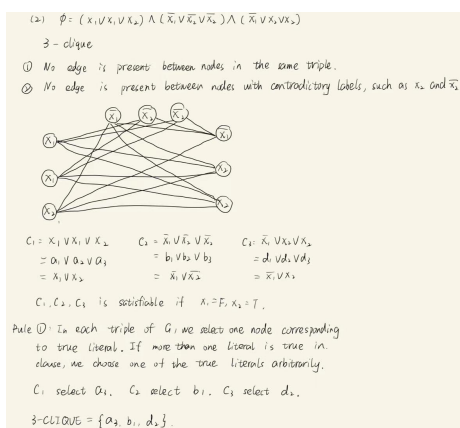


图 5: Vertex-Cover2-2

图 6: Vertex-Cover<sub>3</sub>